Sol 1 — Boot Sequence

for **macOS** — Intel and Apple Silicon

Sol 1 · Boot Sequence

Six parts, about 6-8 hours. Most of that is the environment, not the code — you write about 30 lines.

https://github.com/KrithinP/vanguard-inductions-2026

Every link in this document opens the live page on GitHub.

Before you start

The repository — everything lives here, and you'll work from your own copy:

> **github.com/KrithinP/vanguard-inductions-2026**

Five minutes of setup, on the website and then in a terminal:

1. ** Star **the repo, then click** Fork** (top right) to get your own copy.
2. **Clone it:** `git clone https://github.com/YOUR-USERNAME/vanguard-inductions-2026.git`
3. **Let git talk to GitHub:** run `gh auth login` → *GitHub.com* → *HTTPS* → *browser*. GitHub removed password logins, so without this your first `git push` fails.
4. **Tell git who you are:** `git config --global user.name "Your Name"` and `user.email "you@example.com"`
5. **Turn on Actions in your fork** — Actions tab → "*I understand my workflows, go ahead and enable them*". GitHub disables them on new forks, and until you click it you get no feedback on anything you push.
6. **Pick a callsign** (3–16 characters, A–Z and 0–9, upper case). It's what shows on the progress board instead of your name: `echo "NIGHTJAR" > callsign.txt`

What you need

A laptop with **8 GB RAM** and ~20 GB free, and about **6–8 hours a week**.

No prior Linux, ROS, robotics or Python experience is assumed. This guide starts from *what is a terminal* and writes out every command with the output you should see.

How you'll be judged

Not on how much you finish. In order: can you explain your code, does it work, is it readable, and were you honest about what broke. Shortlisted candidates get a short call where we ask you to walk through your own work and change something in it live.

You may use AI tools. The one rule is that you must understand every line you submit.

When you get stuck — and you will

Open an **Issue** on the repository rather than messaging anyone. Answers are public so everyone benefits, and how well you ask is part of what we're looking at. There's no penalty for asking; there is one for going quiet for two weeks.

Include: what you tried, the exact command, the full error, and the output of `./tools/vanguard doctor`.

Deadline: 21 September 2026, 23:59 IST. One pull request per person, titled `NAME [ID_NUMBER]`, which you keep pushing to as you go.

What’s in here

Work through it in order with a terminal open. Type the commands rather than reading them — that is the whole difference.

PART 1

Your mission

What you're building, and what to hand in

SOL 1 — "Boot Sequence"

```
╔═══════════════════════════════════════════════════╗
║ SOL 001 · LANDING + 0 ║
║ SPROSCAPE · 18.4°N 77.5°E ║
║ STATUS: SYSTEMS OFFLINE ║
╠═══════════════════════════════════════════════════╣
║ Recover the workstation. Wake the flight computer. ║
╚═══════════════════════════════════════════════════╝
```

Objective: bring a workstation online, recover the mission authentication phrase, and put the rover's first command on the wire. **Effort:** 6–8 hours. Spread it over days — do not start this the night before.

Mission briefing

A rover is not one program. It's dozens, running at once, talking to each other over a network — one for the camera, one for the wheels, one deciding where to go. Before you can write any of them, you need the environment they live in, and you need to be able to *see* what they're saying to each other.

That's Sol 1. It's the least glamorous mission you'll fly and the one everything else stands on.

You'll also find the mission directory came back from descent corrupted. Recovering it is how you'll learn the terminal — not by reading a list of commands, but by needing them.

Tasks

1 · Get the workstation running

Install **Ubuntu 24.04 LTS**.

Confirm with `lsb_release -a` — it must say `Release: 24.04`.

2 · Learn to drive the terminal

Read it with a terminal open. Type every command as you go. Reading about `grep` teaches you nothing; using it teaches you `grep`.

3 · Understand `apt`

4 · Understand `.bashrc`, `PATH` and `source`

Do not skim this one. It is the difference between debugging your code and debugging your shell, and almost everyone who skips it loses an evening to `command not found` within the week.

5 · Install ROS 2 Jazzy

Finish with:

```
./tools/vanguard doctor
```

Every line must read **GO** before you continue.

6 · Learn to inspect a running system

Run turtlesim and work through the nine inspection commands. Drive the turtle by hand with `ros2 topic pub` before you write any code.

7 · Recover the mission directory

Somewhere in this repository is a hidden directory holding three fragments of the authentication phrase. Find them.

You'll need: listing hidden files, searching recursively inside files, making a script executable, and extracting an archive. All four are in `handbook/01`.

Start with: `ls -a`
Everything else follows from noticing what's there.

8 · Authenticate

Join the three fragments with hyphens, in order, upper case. Then hash them with your callsign:

```
echo -n "FRAGMENT1-FRAGMENT2-FRAGMENT3:YOURCALLSIGN" | sha256sum | cut -c1-16
```

Write those 16 characters into `FLAG.txt` at the repo root:

```
echo "your16charresult" > FLAG.txt
```

Your callsign must match `callsign.txt` exactly. **The flag is derived from your own callsign — copying someone else's will fail, and we will see that it failed.**

Need the reminder later: `./tools/vanguard flag`

9 · Write your first node

Build a package called **`first_light`** containing a node that:

- publishes **`geometry_msgs/msg/Twist`**
- to **`/turtle1/cmd_vel`**
- at **10 Hz**
- making the turtle draw a **shape** — a circle works; a square is better

Register the entry point as **`circle`** (even if you drew a square — the automated checks look for that name):

```
ros2 run first_light circle
```

Then copy the package into this repository:

```
mkdir -p src/sol1
cp -r ~/vanguard_ws/src/first_light src/sol1/
```

> Copy the **source only**. Never commit `build/`, `install/` or `log/` — `.gitignore` already blocks them.

10 · Write your mission log

Create `MISSION_LOG.md` at the repo root. A few honest paragraphs:

- What did you get working?
- **What broke, and how did you work it out?**
- What still doesn't make sense to you?
- Roughly how long did it take?

The second and third questions are the ones we actually read. "I spent two hours on `command not found` before realising I hadn't sourced" is a *good* entry — it tells us you can diagnose. Nobody believes a log that says everything went fine.

Checklist

Run this before you submit:

```
./tools/vanguard check
```

```
MISSION 1 — BOOT SEQUENCE
 GO callsign = NIGHTJAR
 GO flag submitted (verified at mission control)
 GO package present at src/sol1/first_light
 GO node source present
 GO mission log written
 GO colcon build succeeded
 GO node publishes on /turtle1/cmd_vel

MISSION 2 — ROLLING CHASSIS
 · not started

MISSION 3 — EYES
 · not started

MISSION 4 — THE WORLD MODEL (bonus)
 · not attempted — this costs you nothing

________________________________________
Started: 1 Complete: 1
ALL STATIONS GO.
```

Missions you haven't started are ignored — they can never fail your build. The same check runs automatically on every push to your fork.

File	What it should be
`callsign.txt`	your callsign, upper case
`FLAG.txt`	16 hex characters
`src/sol1/first_light/`	your package (source only)
`MISSION_LOG.md`	honest write-up

Submitting

Commit as you go, then open a Pull Request to this repository's `main`, titled `NAME [ID_NUMBER]`.

Also record a short screen capture — under 90 seconds — showing your turtle drawing its shape, with `ros2 topic hz /turtle1/cmd_vel` visible in a second terminal. Link it in your PR. Don't commit the video file.

In that recording, **say out loud in one sentence why your node uses a timer instead of a `while` loop.** We'd rather hear you explain one thing than watch a polished demo.

Stuck?

1. `./tools/vanguard doctor`
2. `handbook/99-troubleshooting.md`
3. Open an Issue

There is no penalty for asking. There's a real one for going quiet for a week.

One command. Four hundred million kilometres. Make it count.

PART 2

Set up your environment

A full Linux desktop in your browser. This covers tasks 1, 3 and 5 of the mission above - Ubuntu, apt and ROS 2 are already installed, so you can skip those and go straight to task 2.

Docker — run the whole environment in your browser

A complete Ubuntu desktop with ROS 2 Jazzy and Gazebo Harmonic, running in a container, viewed through your web browser. **No dual boot, no VM, no partitioning.**

Does this work on my machine?

Your machine	Works?	Notes
Windows 10 / 11		Docker Desktop with the WSL 2 backend (the default)
macOS, Apple Silicon (M1–M4)		Runs **natively** on arm64 — no emulation
macOS, Intel		
Linux (any distro)		Docker Engine + Compose
Ubuntu 24.04 natively	—	You don't need this. Install ROS directly.

The base image is multi-architecture, so Docker pulls the right build for your CPU automatically. **You do not need Ubuntu, and you do not need to touch your disk partitions.**

Requirements: **8 GB RAM** (16 GB comfortable), **~15 GB free disk**, and a browser.

Setting it up

1. Install Docker

- Windows / macOS: Docker Desktop
- Linux: `sudo apt install docker.io docker-compose-v2` (then `sudo usermod -aG docker $USER` and log out and back in)

On Windows, Docker Desktop will ask to enable WSL 2. Say yes.

2. Start it

```
cd docker
docker compose up -d --build
```

The first run builds the image. **Expect 10–20 minutes** — it's downloading a full Linux desktop plus ROS. It only happens once. Later starts take seconds.

3. Open the desktop

Go to **http://localhost:8080** in your browser.

That's a real Ubuntu desktop. Right-click it for a menu; there's a terminal in there.

4. Check it works

In a terminal on that desktop:

```
ros2 run turtlesim turtlesim_node
```

A window with a turtle should appear. If it does, you're ready.

Day-to-day use

Get a terminal without using the browser desktop:

```
docker compose exec vanguard bash
```

Stop it (your work is kept):

```
docker compose down
```

Where your files live

The whole repository is mounted at `/root/vanguard_ws` inside the container. So `~/vanguard_ws` in the container and your cloned repo on your laptop are the same files — the handbook, the tooling, `src/`, everything.

Edit files in your normal editor on your own machine — VS Code, whatever you like — and build and run them inside the container. Changes appear instantly on both sides, in both directions.

That also means the checks work from inside the container:

```
cd ~/vanguard_ws
./tools/vanguard doctor
./tools/vanguard check
```

Anything you create **outside** `~/vanguard_ws` in the container is lost when the container is removed. Keep everything in there.

Honest limitations

- **Simulation will be slower** than a native install. The container renders in software with no direct access to your graphics card. For the worlds in this induction that's fine; it is not what you'd use for serious work.
- **Sound doesn't work.** You don't need it.
- If your laptop has 8 GB RAM total, close Chrome before running the simulator.

Using this instead of installing Ubuntu

Perfectly acceptable. Two things to know:

1. **You still have to read `handbook/00` and `handbook/04`.** The container already has everything installed, so you can skip the *commands* — but the concepts, especially `source` and `.bashrc`, come up in the walkthrough and the container will not answer for you.
2. **Say so in your `MISSION_LOG.md`.** It's context for us, not a deduction. Nobody is marked down for using this.

If it goes wrong

`docker: command not found` — Docker isn't installed, or on Windows you're in a terminal that can't see it. Use PowerShell or the WSL terminal.

Docker Desktop won't start at all (Windows) — nearly always one of two things:

1. **Virtualization is disabled in your BIOS.** Reboot into BIOS (usually `F2` or `Del`), find *Intel VT-x*, *AMD-V* or *SVM Mode*, and enable it. Laptops ship with this off surprisingly often.
2. **WSL 2 isn't installed.** In PowerShell as administrator: `powershell wsl --install` Then reboot.

`no space left on device` partway through — Docker images are big.

```
docker system prune -a
```

`permission denied while trying to connect to the Docker daemon` (Linux) —

```
sudo usermod -aG docker $USER
```

Then log out and back in completely.

Port 8080 already in use — change the left-hand number in `docker-compose.yml`, e.g. `"8081:80"`, then use `localhost:8081`.

Desktop is blank, grey, or won't load — give it a minute after `up`. Then check `docker compose ps`. If it's restarting, you're probably out of memory — raise Docker's RAM limit (Docker Desktop → Settings → Resources → 8 GB+).

Build fails partway — usually a dropped network connection. Just run `docker compose up -d --build` again; it resumes from where it stopped.

Everything is unbearably slow — check Docker Desktop's memory allocation, and close other applications. If your machine has 4 GB RAM, this route won't be comfortable; talk to us.

Still stuck? Open an Issue with your OS, the exact command, and the full error.

PART 3

The terminal

Linux fundamentals, and recovering the mission directory

01 — The terminal

Time: 45 minutes. **By the end:** you can move around your filesystem, read files, search inside them, and change what a file is allowed to do — without touching a mouse.

Open a terminal with `Ctrl+Alt+T`. You'll see something like:

```
krithin@rover-lab:~$
```

That's the **prompt**: your username, your machine name, then where you currently are, then `$`. The `~` means your home directory (`/home/yourname`).

Robotics happens here. Not because we're purists — because a rover 500 m away over a flaky radio link has no desktop. Every tool you'll use for the rest of your time on this team is driven from a prompt like this one.

Where am I, and what's here

```
pwd
```

```
/home/krithin
```

`pwd` = *print working directory*. It answers "where am I".

```
ls
```

```
Desktop Documents Downloads Music Pictures Public Templates Videos
```

`ls` = *list*. But it's hiding things:

```
ls -a
```

```
. .. .bashrc .cache .config .profile Desktop Documents Downloads ...
```

Any file whose name starts with `.` is hidden from plain `ls`. That's the whole rule — there's no special "hidden" attribute like Windows has. `-a` means *all*.

Remember this one. Part of Sol 1 depends on it.

Add `-l` for the long view:

```
ls -la
```

```
drwxr-xr-x 18 krithin krithin 4096 Aug 30 14:02 .
drwxr-xr-x 3 root root 4096 Aug 12 09:11 ..
-rw-r--r-- 1 krithin krithin 3771 Aug 12 09:11 .bashrc
drwxr-xr-x 2 krithin krithin 4096 Aug 30 13:55 Desktop
```

Reading a line: `d` at the start means directory, `-` means regular file. The next nine characters are permissions (section below). Then owner, group, size, date, name.

`.` is *this directory*. `..` is *the one above*.

Moving around

```
cd Downloads # go into Downloads
pwd # /home/krithin/Downloads
cd .. # go back up one level
cd # go home from anywhere
cd - # go back to where you just were
```

Press `Tab` constantly. Type `cd Dow` then hit `Tab` — the shell completes it. Hit `Tab` twice to see all the options. This is not a nicety; experienced people type maybe half the characters you do, and make far fewer typos.

Absolute vs relative paths

- **Absolute** starts with `/`: `/home/krithin/Downloads`. Always means the same place, from anywhere.
- **Relative** doesn't: `Downloads`, `../Pictures`. Means something different depending on where you are.

Most "file not found" errors in this induction are a relative path run from the wrong directory. When something can't be found, run `pwd` first.

Making, copying, moving, deleting

```
mkdir practice # make a directory
cd practice
touch notes.txt # create an empty file
cp notes.txt backup.txt # copy
mv backup.txt old.txt # rename (move is rename)
rm old.txt # delete
cd ..
rm -r practice # delete a directory and everything in it
```

> **Warning —** `rm` does not use a recycle bin.** There is no undo. `rm -rf` on the wrong path is how people lose a semester of work. Run `ls` on a path before you `rm` it. Never run `rm -rf /` or `rm -rf ~` — read any command someone gives you before pasting it.

Reading files

```
cat notes.txt # dump the whole file
head -20 notes.txt # first 20 lines
tail -20 notes.txt # last 20 lines
less notes.txt # scroll it (arrows to move, q to quit)
```

`tail` is the one you'll live in — errors are always at the bottom.

Finding things

```
find . -name "*.log" # every .log file below here
find . -type d -name "src" # every directory called src
```

Searching inside files

This is the single most useful command in this document.

```
grep "error" robot.log # lines containing "error"
grep -i "error" robot.log # ignore upper/lower case
grep -n "error" robot.log # show line numbers
grep -r "error" logs/ # search every file under logs/
grep -rn "FRAGMENT" . # recursive, with line numbers, from here
```

`grep -r` searches a whole directory tree. When you know *what* you're looking for but not *which file* it's in, this is the answer.

Pipes — connecting commands

The `|` character takes one command's output and feeds it to the next.

```
ls -la | grep "Aug" # only lines mentioning Aug
cat robot.log | wc -l # count the lines
history | grep colcon # which colcon commands have I run before?
```

Chain as many as you like:

```
cat robot.log | grep -i error | tail -5
```

Read the log, keep only error lines, show the last five.

Permissions and `chmod`

Back to that `-rw-r--r--`. After the first character, it's three groups of three:

```
 rw- r-- r--
owner group everyone
```

`r` read, `w` write, `x` execute. A script **cannot be run unless it has `x`**, no matter what's inside it.

```
ls -l script.sh
```

```
-rw-r--r-- 1 krithin krithin 117 Aug 30 15:37 script.sh
```

```
./script.sh
```

```
bash: ./script.sh: Permission denied
```

Fix it:

```
chmod +x script.sh
./script.sh
```

`chmod +x` = *change mode, add execute*. You'll do this constantly.

> **`./` matters.** `script.sh` alone tells the shell to look for an installed program of that name. `./script.sh` says "the one in this directory, right here".

Archives

```
tar czf stuff.tar.gz mydir/ # compress a directory
tar xzf stuff.tar.gz # extract it
tar tzf stuff.tar.gz # list contents without extracting
```

Remember it as **e**x**t**ract = `x`, **c**reate = `c`.

Hashing

```
echo -n "hello" | sha256sum
```

```
2cf24dba5fb0a30e26e83b2ac5b9e29e1b161e5c1fa7425e73043362938b9824 -
```

A hash is a fixed-length fingerprint of some input. Same input, same hash, always. `-n` on `echo` means *don't add a newline* — with it you'd get a different hash, which is exactly the kind of detail that bites people.

Trim it with `cut`:

```
echo -n "hello" | sha256sum | cut -c1-16
```

```
2cf24dba5fb0a30e
```

`cut -c1-16` = keep characters 1 to 16.

Cheat sheet

Need	Command
Where am I	`pwd`
What's here (incl. hidden)	`ls -la`
Go somewhere	`cd path`
Go home / go back	`cd` / `cd -`
Make a directory	`mkdir name`
Read a file	`cat head tail less`
Find a file by name	`find . -name "pattern"`
Find text inside files	`grep -rn "text" .`
Make runnable	`chmod +x file`
Extract an archive	`tar xzf file.tar.gz`
Fingerprint some text	`echo -n "x" | sha256sum`
Stop a running program	`Ctrl+C`
Clear the screen	`clear`

You now have everything you need for the `.mission/` directory in Sol 1. Go and try it before continuing — it's more fun than reading.

Next: `02-apt.md`.

PART 4

How ROS finds things

source, PATH and .bashrc - the one that catches everyone

03 — `.bashrc`, `PATH`, and why `source` exists

Time: 30 minutes. Read this one properly. **By the end:** you'll understand the error that costs every ROS beginner an evening, and you'll never lose that evening.

Here is the single most common exchange in robotics:

> "`ros2: command not found`." "Did you source it?" "...what?"

This page is that answer. Everything below builds to it.

Environment variables

Every running program has a set of named values attached to it — its **environment**.

```
echo $HOME
```

```
/home/krithin
```

`$` means *the value of*. Set one yourself:

```
MY_ROVER=perseverance
echo $MY_ROVER
```

```
perseverance
```

Now — and this is the important part — **open a new terminal and try again**:

```
echo $MY_ROVER
```

```

```

Empty. The variable belonged to that one terminal. It died with it.

> **Every terminal window is a separate world.** Things you set in one do not exist in another. Hold on to this; it explains almost everything that follows.

`PATH`

```
echo $PATH
```

```
/usr/local/bin:/usr/bin:/bin:/usr/games
```

A colon-separated list of directories. When you type `ls`, the shell walks that list looking for a program called `ls` and runs the first match.

If a program isn't in a `PATH` directory, typing its name gives `command not found` — even if it's installed and sitting right there on disk.

That's the whole mystery. `ros2` is installed at `/opt/ros/jazzy/bin/ros2`, and `/opt/ros/jazzy/bin` is not in your `PATH` by default.

`source`

ROS ships a script that sets up your environment:

```
source /opt/ros/jazzy/setup.bash
```

It adds ROS to `PATH`, sets `ROS_DISTRO=jazzy`, and sets several other variables ROS needs to find its own packages.

Check:

```
echo $ROS_DISTRO
which ros2
```

```
jazzy
/opt/ros/jazzy/bin/ros2
```

Why `source` and not `./setup.bash`

This distinction is worth thirty seconds.

- `./setup.bash` runs the script in a *new* shell. It sets variables in that new shell, which then exits, taking them with it. **Nothing changes for you.**
- `source setup.bash` runs it in your *current* shell. The variables land in the terminal you're sitting in. **This is what you want.**

`.` is shorthand for `source`, so `. setup.bash` is the same thing. You'll see both.

> **What just happened**
>
> `source` didn't install anything. ROS was already on disk. It edited a few variables in this one terminal so the shell can now find what was always there.

And then you open a new terminal

```
ros2 --version
```

```
bash: ros2: command not found
```

Of course. New terminal, new environment. The `source` you ran applied to the old one.

`.bashrc` — the fix

`~/.bashrc` is a script that runs **automatically every time you open a terminal**. Put the `source` line in there and every new terminal arrives ready.

```
echo "source /opt/ros/jazzy/setup.bash" >> ~/.bashrc
```

> **Warning —** `>>` appends. `>` overwrites.** `>` would erase your entire `.bashrc`. One character. Get it right.

Apply it to the terminal you're in right now:

```
source ~/.bashrc
```

Or just open a new terminal. From here on, every terminal has ROS.

Look at it

```
cat ~/.bashrc | tail -5
```

Or edit it:

```
nano ~/.bashrc
```

Nano's controls are along the bottom. `^O` means `Ctrl+O` (save), `^X` is exit.

Diagnosing this yourself, forever

When a ROS command isn't found:

```
echo $ROS_DISTRO # empty? → not sourced
which ros2 # nothing? → not on PATH
ls /opt/ros/jazzy # missing? → not installed at all
```

Three commands, in that order. They separate "not installed" from "not sourced" — two completely different problems that produce an identical error message.

Later: two things to source

Once you build your own packages you'll have a second setup file:

```
source /opt/ros/jazzy/setup.bash # ROS itself
source ~/vanguard_ws/install/setup.bash # your own packages
```

Order matters — ROS first, yours second. Yours *overlays* ROS's. We'll come back to this in `06-workspace-and-packages.md`.

The one-line summary

Installed ≠ available. Installing puts files on disk. `source` tells *this terminal* where they are. `.bashrc` does that automatically for every future terminal.

Next: `04-install-ros2.md`.

PART 5

Nodes, topics, and your first program

Inspecting a robot, then writing one

05 — Nodes, topics and messages

Time: 45 minutes. **By the end:** you can inspect a running robot without reading a line of its code.

The idea

A robot is not one program. It's dozens of small ones running at once: one talking to the camera, one to the motors, one planning where to go, one watching the battery.

In ROS each of those is a **node**. Nodes don't call each other. They publish messages onto named channels called **topics**, and anything interested subscribes.

```
[camera_node] ──publishes──> /camera/image_raw ──> [detector_node]
└──> [recorder_node]
```

The camera node doesn't know the detector exists. That's the point:

- You can **restart one node** without restarting the robot.
- You can **subscribe to any topic** to see exactly what a node is emitting.
- You can **swap the real camera for a simulated one** and nothing downstream notices.

That last one is why a rover can be developed entirely in simulation and then run on real hardware unchanged.

Get something running

Terminal 1:

```
ros2 run turtlesim turtlesim_node
```

A window with a turtle. Leave it running.

Nodes

Terminal 2:

```
ros2 node list
```

```
/turtlesim
```

```
ros2 node info /turtlesim
```

```
/turtlesim
 Subscribers:
 /turtle1/cmd_vel: geometry_msgs/msg/Twist
 Publishers:
 /turtle1/pose: turtlesim/msg/Pose
 /rosout: rcl_interfaces/msg/Log
 Service Servers:
 /clear: std_srvs/srv/Empty
 /spawn: turtlesim/srv/Spawn
 ...
```

You just read a node's entire interface without opening its source. **Subscribers** = what it listens to. **Publishers** = what it emits.

The important line: it subscribes to `/turtle1/cmd_vel` expecting `geometry_msgs/msg/Twist`. To move the turtle, publish that.

Topics

```
ros2 topic list
```

```
/parameter_events
/rosout
/turtle1/cmd_vel
/turtle1/color_sensor
/turtle1/pose
```

`echo` — watch the traffic

```
ros2 topic echo /turtle1/pose
```

```
x: 5.544444561004639
y: 5.544444561004639
theta: 0.0
linear_velocity: 0.0
angular_velocity: 0.0
---
```

Live data, streaming. `Ctrl+C` to stop.

`ros2 topic echo` is your primary debugging tool. When something misbehaves, you don't add print statements — you `echo` the topic and see what's actually on it.

`info` — who's connected

```
ros2 topic info /turtle1/cmd_vel
```

```
Type: geometry_msgs/msg/Twist
Publisher count: 0
Subscriber count: 1
```

`Publisher count: 0` means nobody is sending. If a robot isn't moving, this number tells you instantly whether the problem is upstream (nothing publishing) or downstream (publishing fine, motors ignoring it).

`hz` — how fast

```
ros2 topic hz /turtle1/pose
```

```
average rate: 62.456
    min: 0.015s max: 0.017s std dev: 0.00051s window: 64
```

Rate matters. A camera meant to run at 30 Hz delivering 4 Hz is a real fault, and this is how you catch it.

`pub` — send a message by hand

```
ros2 topic pub --rate 1 /turtle1/cmd_vel geometry_msgs/msg/Twist \
 "{linear: {x: 2.0}, angular: {z: 1.8}}"
```

Watch the turtle move in a circle. `Ctrl+C` to stop.

You just drove a robot from the command line. In Sol 1 you'll write a node that does this properly — but being able to do it by hand is how you test whether a problem is in your code or in the robot.

Messages

```
ros2 interface show geometry_msgs/msg/Twist
```

```
Vector3 linear
    float64 x
    float64 y
    float64 z
Vector3 angular
    float64 x
    float64 y
    float64 z
```

A `Twist` is two 3-D vectors: **linear** velocity (m/s) and **angular** velocity (rad/s). For a ground robot that drives forward and turns, only `linear.x` and `angular.z` do anything — it can't slide sideways or fly.

> **Conventions worth learning now (REP-103/105):** x is forward, y is left, z is up. Angles in radians, distances in metres, counter-clockwise positive. ROS is strict about this and it eliminates a whole class of bug.

See the whole system

```
ros2 run rqt_graph rqt_graph
```

A live diagram of every node and topic. With `ros2 topic pub` still running you'll see it drawn as a box feeding `/turtle1/cmd_vel` into `/turtlesim`.

When you can't work out why two things aren't talking, this picture usually shows you in one glance — most often a topic name typo, which appears as two boxes with no line between them.

The toolkit

Question	Command
What's running?	`ros2 node list`
What does this node do?	`ros2 node info /name`
What channels exist?	`ros2 topic list`
What's on this channel?	`ros2 topic echo /name`
Is anyone publishing?	`ros2 topic info /name`
How fast?	`ros2 topic hz /name`
Send one by hand	`ros2 topic pub /name type "{...}"`
What's in this message?	`ros2 interface show <type>`
Show me the picture	`rqt_graph`

Learn these nine. They diagnose most problems you'll hit for the next three years.

Next: `06-workspace-and-packages.md`.

06 — Workspaces and packages

> **Time:** 30 minutes. **By the end:** you have your own ROS 2 package, built, and visible to `ros2 run`.

Workspaces

A **workspace** is a directory where you build your own ROS code. It has one rule: your source goes in `src/`, and the build tool creates everything else.

```
mkdir -p ~/vanguard_ws/src
cd ~/vanguard_ws
```

`-p` creates parent directories as needed. After building you'll have:

```
vanguard_ws/
├── src/ ← your code. The only directory you write in.
├── build/ ← intermediate build files (generated)
├── install/ ← the finished, runnable result (generated)
└── log/ ← build logs (generated)
```

> **Never commit `build/`, `install/` or `log/` to git.** They're generated, enormous, and machine-specific. Our `.gitignore` already excludes them — if you ever see them in `git status`, something is wrong.

Packages

A **package** is one unit of ROS code — it has a name, declares its dependencies, and can be built and run. Everything in ROS is a package, including everything you installed with apt.

Create one:

```
cd ~/vanguard_ws/src
ros2 pkg create first_light \
 --build-type ament_python \
 --dependencies rclpy geometry_msgs \
 --license Apache-2.0
```

Breaking that down: - `first_light` — the package name. Lower case, underscores, no hyphens. ROS is strict about this and the error when you get it wrong is not obvious. - `--build-type ament_python` — a Python package. (C++ would be `ament_cmake`.) - `--dependencies rclpy geometry_msgs` — `rclpy` is the ROS 2 Python library; `geometry_msgs` has `Twist`. Declaring them here writes them into `package.xml`. - `--license Apache-2.0` — optional, but omitting it prints a long warning.

> **Warning —** Put the package name first.** `--dependencies` accepts any number of values, so if you write it before the name, it swallows the name as a dependency and you get `error: the following arguments are required: package_name`. Either put the name first as above, or put `--dependencies` last.

You get:

```
first_light/
├── first_light/ ← your Python code goes HERE
│ └── __init__.py
├── resource/first_light
├── test/
├── package.xml ← name, version, dependencies
├── setup.py ← how to build and install it
└── setup.cfg
```

> **Yes, `first_light/first_light/`.** The outer is the package directory; the inner is the Python module. Everyone finds this odd. Your code goes in the inner one.

Building

Always build from the **workspace root**, never from inside a package:

```
cd ~/vanguard_ws
colcon build
```

```
Starting >>> first_light
Finished <<< first_light [0.94s]

Summary: 1 package finished [1.31s]
```

Useful variations:

```
colcon build --symlink-install # edit Python without rebuilding
colcon build --packages-select first_light # build just one package
```

`--symlink-install` links to your source instead of copying it, so editing a Python file takes effect immediately. **Use it — it'll save you a hundred rebuilds.**

Sourcing your workspace

Building doesn't make ROS aware of your package. You have to source it:

```
source ~/vanguard_ws/install/setup.bash
```

Now `ros2 run first_light ...` works. Same lesson as `03`: building put files on disk, sourcing told this terminal where they are.

Make it automatic:

```
echo "source ~/vanguard_ws/install/setup.bash" >> ~/.bashrc
```

Your `~/.bashrc` should now end with **two** source lines, in this order:

```
source /opt/ros/jazzy/setup.bash
source ~/vanguard_ws/install/setup.bash
```

ROS first, yours second — yours *overlays* ROS's, so your packages can override system ones.

> **Warning —** Chicken and egg.** If `install/setup.bash` doesn't exist yet, that second line prints an error in every new terminal. Build once first, or ignore the complaint until you have.

The rebuild ritual

You will do this hundreds of times. Learn it now:

```
cd ~/vanguard_ws
colcon build --symlink-install
source install/setup.bash
ros2 run first_light <node>
```

When something you *know* you fixed still misbehaves, 90% of the time you either didn't rebuild or didn't re-source. Check that before you debug anything else.

If it went wrong

`colcon: command not found`

```
sudo apt install -y python3-colcon-common-extensions
```

`Package 'first_light' not found` Not sourced, or the build failed. Run `colcon build` again and read the output — `Finished` means good, `Failed` means read the error above it.

Build succeeds but `ros2 run` says no executable Your entry point isn't registered in `setup.py`. That's the next page.

Build says `Finished` but `ros2 run` says `Package 'first_light' not found` Look near the top of the build output for:

```
WARNING: Failed to parse ROS package manifest ... Invalid email "..." for person
```

`ros2 pkg create` copies your **git email** into `package.xml` as the maintainer. If that address is malformed, colcon *warns*, reports success anyway, and quietly builds something ROS can never find. Open `package.xml`, fix the `<maintainer email="...">` line to a real address, and rebuild. Then fix it at source so it doesn't recur:

```
git config --global user.email "your.real@email.com"
```

`./tools/vanguard doctor` now checks this for you.

Weird errors after renaming or moving things Stale build artefacts. Nuke and rebuild:

```
cd ~/vanguard_ws && rm -rf build install log && colcon build --symlink-install
```

`SetuptoolsDeprecationWarning` Noise, not an error. Ignore it.

Next: `07-first-node.md` — where you finally write something.

07 — Writing your first node

> **Time:** 45 minutes. **By the end:** you understand every line of a ROS 2 publisher, and you're ready to write your own.

This page walks through a **different** node than the one you have to submit. Read it, run it, understand it — then write yours. Copying this one won't get you there, and it isn't meant to.

The example: a node that says hello

Create the file:

```
cd ~/vanguard_ws/src/first_light/first_light
nano hello.py
```

```
import rclpy # the ROS 2 Python library
from rclpy.node import Node # base class for every node
from std_msgs.msg import String # the message type we'll send

class HelloPublisher(Node):
 def __init__(self):
 super().__init__('hello_publisher') # this node's name on the graph

 # Create a publisher: (message type, topic name, queue depth)
 self.publisher_ = self.create_publisher(String, 'greeting', 10)

 # Call self.tick() every 0.5 s — that's 2 Hz
 self.timer = self.create_timer(0.5, self.tick)
 self.count = 0

 self.get_logger().info('Hello publisher started')

 def tick(self):
 msg = String()
 msg.data = f'hello from the rover, message {self.count}'
 self.publisher_.publish(msg)
 self.get_logger().info(f'Publishing: "{msg.data}"')
 self.count += 1

def main(args=None):
 rclpy.init(args=args) # start up ROS
 node = HelloPublisher()
 try:
 rclpy.spin(node) # hand control to ROS; run callbacks forever
 except KeyboardInterrupt:
 pass
 finally:
 node.destroy_node()
 rclpy.shutdown()

if __name__ == '__main__':
 main()
```

Line by line

`class HelloPublisher(Node)` — your node inherits from `Node`, which gives you `create_publisher`, `create_timer`, `get_logger` and the rest.

`super().__init__('hello_publisher')` — registers this name on the ROS graph. It's what shows up in `ros2 node list`.

`create_publisher(String, 'greeting', 10)` — message type, topic name, and **queue depth**. The queue holds messages if a subscriber is slow. 10 is a normal default.

`create_timer(0.5, self.tick)` — ROS calls `self.tick` every 0.5 seconds.

> **Why a timer and not `while True:` with `sleep`?** A `while` loop blocks the node — it can't process incoming messages or respond to shutdown while sleeping. A timer hands control back to ROS between calls, so everything else keeps working. This is the single most common beginner mistake in ROS. Use timers.

`rclpy.spin(node)` — gives control to ROS. It sits there dispatching callbacks until you `Ctrl+C`. Nothing after `spin` runs until the node shuts down.

`self.get_logger().info(...)` — ROS's `print`. It carries timestamps and node names, and it works when your node is running on a rover instead of your laptop. Use it, not `print()`.

Registering the entry point

Writing the file isn't enough — ROS has to know it's runnable. Open `setup.py`:

```
nano ~/vanguard_ws/src/first_light/setup.py
```

Find `entry_points` and add a line:

```
entry_points={
 'console_scripts': [
 'hello = first_light.hello:main',
 ],
},
```

The format is `command_name = package.module:function`: - `hello` — what you'll type after `ros2 run first_light` - `first_light.hello` — the package, then the file (no `.py`) - `main` — the function to call

> **This is the step everyone forgets.** If `ros2 run` says "*No executable found*", you either didn't add this line or didn't rebuild after.

Build and run

```
cd ~/vanguard_ws
colcon build --symlink-install
source install/setup.bash
ros2 run first_light hello
```

```
[INFO] [1730294412.583] [hello_publisher]: Hello publisher started
[INFO] [1730294413.084] [hello_publisher]: Publishing: "hello from the rover, message 0"
[INFO] [1730294413.584] [hello_publisher]: Publishing: "hello from the rover, message 1"
```

In a second terminal, prove it's real:

```
ros2 topic echo /greeting
ros2 topic hz /greeting # should read ~2.0
```

> **What just happened**
> You wrote a program that broadcasts on a named channel. It doesn't know or care whether anything is listening. Another program on another machine could subscribe and neither would need reconfiguring. That is the whole architecture.

Now go and write yours

Your first task needs a node that publishes **`geometry_msgs/msg/Twist`** to **`/turtle1/cmd_vel`** at **10 Hz**, making the turtle draw a shape.

Things to work out for yourself:

1. **Different import.** `from geometry_msgs.msg import Twist` — not `String`.
2. **Different fields.** Run `ros2 interface show geometry_msgs/msg/Twist` and look. It's nested: `msg.linear.x`, `msg.angular.z`.
3. **10 Hz.** What timer period gives 10 Hz?
4. **A shape.** A circle is constant `linear.x` and constant `angular.z` — that's the easy one. A square needs you to *change* the velocities over time: drive, stop, turn, repeat. Squares are more interesting and score better.
5. **Name the entry point `circle`** so the automated checks can find it. Yes, even if you drew a square.

Check `ros2 topic hz /turtle1/cmd_vel` really shows ~10.

If it went wrong

`No executable found` `setup.py` entry point missing or misspelled, or you didn't rebuild. Do both.

`ModuleNotFoundError: No module named 'first_light'` Didn't source `install/setup.bash` in this terminal.

Node runs, turtle doesn't move Check the topic name. `ros2 topic info /turtle1/cmd_vel` — if `Publisher count` is 0, your node is publishing somewhere else. A leading `/` matters: `'turtle1/cmd_vel'` and `'/turtle1/cmd_vel'` resolve differently depending on namespace. Use the leading slash.

`AttributeError: 'Twist' object has no attribute 'x'` It's `msg.linear.x`, not `msg.x`. Look at `ros2 interface show` again.

Turtle moves once and stops You published in `__init__` instead of from a timer callback. One message, one move.

Turtle spirals off and hits the wall Expected — `turtlesim` has walls. Reset with `ros2 service call /reset std_srvs/srv/Empty`.

That's the fundamentals. Go back to your sol sheet and finish it.

PART 6

When it breaks

Every error in this guide, and what fixes it

99 — Troubleshooting index

Every error in this handbook, in one place. `Ctrl+F` for your error text.

Before anything else

```
./tools/vanguard doctor
```

It checks the whole environment and prints the fix for each failure. Run it first, every time. When you open an Issue, paste its output.

The three-command diagnosis

When a ROS command isn't found, run these in order:

```
echo $ROS_DISTRO # empty → not sourced
which ros2 # nothing → not on PATH
ls /opt/ros/jazzy # missing → not installed
```

They separate "not installed" from "not sourced" — two different problems with the same error message.

Errors

Error	Cause	Fix
`Support for password authentication was removed`	GitHub killed password auth	`gh auth login` → HTTPS → browser · README
`bad interpreter: /bin/bash^M`	Windows line endings (CRLF)	`git config --global core.autocrlf input`, then re-clone
Actions tab says workflows are disabled	GitHub disables them on new forks	Click "*I understand my workflows...*" on **your fork's** Actions tab
`Permission denied (publickey)` on push	No SSH key, or wrong remote	Easiest: `gh auth login` and use HTTPS
Docker Desktop won't start (Windows)	Virtualization off in BIOS, or WSL 2 missing	Enable *Intel VT-x / AMD-V* in BIOS; `wsl --install` in PowerShell
`no space left on device` mid-build	Docker images are large	`docker system prune -a` frees old layers
`ros2: command not found`	Not sourced	`source /opt/ros/jazzy/setup.bash` · 03
`Unable to locate package ros-jazzy-*`	Repo not added, or no `apt update`	04 step 3
`curl: (35) Connection reset` on the ROS key	Flaky network to GitHub	**Just retry** — often works on the 3rd or 4th go · 04
`Author identity unknown` / can't commit	git doesn't know who you are	`git config --global user.name "..."` and `user.email`
`GPG error` / `NO_PUBKEY`	Signing key missing	Redo the `curl` in 04 step 3
`Could not get lock /var/lib/dpkg/`	Another apt is running	Wait, then 02
`externally-managed-environment`	Ubuntu 24.04 pip policy	Add `--user --break-system-packages`
`colcon: command not found`	Not installed	`sudo apt install python3-colcon-common-extensions`
`ros2 pkg create: error: ... required: package_name`	`--dependencies` swallowed the name	Put the package name first · 06
`Package 'x' not found`	Not built or not sourced	`colcon build` then `source install/setup.bash`
Build says success but `Package 'x' not found`	Invalid maintainer email in `package.xml`	colcon only *warns*. Fix the email in `package.xml` · 06
`No executable found`	Missing `setup.py` entry point	07
`ModuleNotFoundError: No module named`	Workspace not sourced	`source ~/vanguard_ws/install/setup.bash`
`AttributeError: 'Twist' has no attribute 'x'`	It's nested	`msg.linear.x` · 07
`Permission denied` running a script	No execute bit	`chmod +x script.sh` · 01
Talker/listener can't hear each other	Another ROS on the network	`export ROS_DOMAIN_ID=42` in both
Node runs, robot doesn't move	Wrong topic	`ros2 topic info <topic>` — check `Publisher count`
Turtle moves once and stops	Published in `__init__`, not a timer	07
Fixed it but nothing changed	Didn't rebuild or re-source	`colcon build && source install/setup.bash`
Strange errors after renaming files	Stale build artefacts	`rm -rf build install log && colcon build`
No boot menu after installing Ubuntu	Secure Boot / Fast Startup	00
Wi-Fi dead in Ubuntu	Broadcom driver	`sudo apt install bcmwl-kernel-source`
Out of disk during install	Partition too small	`sudo apt clean && sudo apt autoremove`

The nuclear option

When a workspace is behaving impossibly:

```
cd ~/vanguard_ws
rm -rf build install log
colcon build --symlink-install
source install/setup.bash
```

This is safe — those three directories are all generated. It fixes a surprising number of problems.

Still stuck

Open an Issue. Include:

1. What you were trying to do
2. The **exact** command you ran
3. The **full** error, not a summary
4. Output of `./tools/vanguard doctor`
5. What you already tried

Questions go in Issues, not DMs — so everyone benefits from the answer, and so we answer it once. **How well you ask is something we're evaluating.** A good question is a strong signal; "it doesn't work" is also a signal.